Python Engineering - Class 04 Reference Guide

Funciones y descomposicion

Guía de consulta de sintaxis y herramientas utilizadas en esta clase. Incluye qué hace cada elemento, cómo se escribe, qué recibe o produce y cuándo conviene utilizarlo.

Mapa rápido

Concepto	Sintaxis	Idea
def	def name(parameters):	Define una función y crea un objeto función con un nombre.
Parámetros	def f(x, y):	Nombres declarados en la definición de la función para recibir información.
Argumentos	f(2, 3)	Valores concretos entregados a una función cuando se realiza la llamada.
return	return value	Devuelve un resultado al código que llamó a la función y finaliza esa ejecución de la func
Ámbito local	local_var	Los nombres creados dentro de una función pertenecen normalmente a esa llamada y n
Argumentos por defecto	def f(g=9.81):	Proporcionan un valor predeterminado cuando el caller no entrega ese argumento.
Múltiples valores de retorno	return h, v, t	Python puede devolver varios valores; conceptualmente se empaquetan en una tuple.
Guarda booleana	if not valid: return ...	Comprobación temprana que evita continuar con datos inválidos y simplifica el flujo.
Llamada de función	result = f(...)	Los paréntesis ejecutan la función; el valor devuelto puede guardarse en una variable.

1. def

Sintaxis: `def name(parameters):`

Define una función y crea un objeto función con un nombre.

```
def dynamic_pressure(rho, speed):
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

2. Parámetros

Sintaxis: `def f(x, y):`

Nombres declarados en la definición de la función para recibir información.

```
def euler_step(h, v, t, g, dt):
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

3. Argumentos

Sintaxis: `f(2, 3)`

Valores concretos entregados a una función cuando se realiza la llamada.

```
h_new, v_new, t_new = euler_step(h, v, t, g, dt)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

4. return

Sintaxis: `return value`

Devuelve un resultado al código que llamó a la función y finaliza esa ejecución de la función.

```
return h_new, v_new, t_new
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

5. Ámbito local

Sintaxis: `local_var`

Los nombres creados dentro de una función pertenecen normalmente a esa llamada y no al código exterior.

```
def f(x):
    local_value = x**2
    return local_value
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

6. Argumentos por defecto

Sintaxis: `def f(g=9.81):`

Proporcionan un valor predeterminado cuando el caller no entrega ese argumento.

```
def initial_energy(h0, v0, g=9.81):
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

7. Múltiples valores de retorno

Sintaxis: `return h, v, t`

Python puede devolver varios valores; conceptualmente se empaquetan en una tuple.

```
return max_h, v_f, t_f
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

8. Guarda booleana

Sintaxis: `if not valid: return ...`

Comprobación temprana que evita continuar con datos inválidos y simplifica el flujo.

```
if not validate_inputs(...):
    return None
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

9. Llamada de función

Sintaxis: `result = f(...)`

Los paréntesis ejecutan la función; el valor devuelto puede guardarse en una variable.

```
result = dynamic_pressure(1.225, 100.0)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

Errores comunes

- Confundir el nombre de una función con su llamada usando paréntesis.
- No revisar tipos, índices o shapes antes de una operación.
- Usar una herramienta porque funciona sin poder explicar qué recibe y qué devuelve.

Chuleta mental

```
def: def name(parameters):
Parámetros: def f(x, y):
Argumentos: f(2, 3)
return: return value
Ámbito local: local_var
Argumentos por defecto: def f(g=9.81):
Múltiples valores de retorno: return h, v, t
Guarda booleana: if not valid: return ...
Llamada de función: result = f(...)
```